Lecture # 2

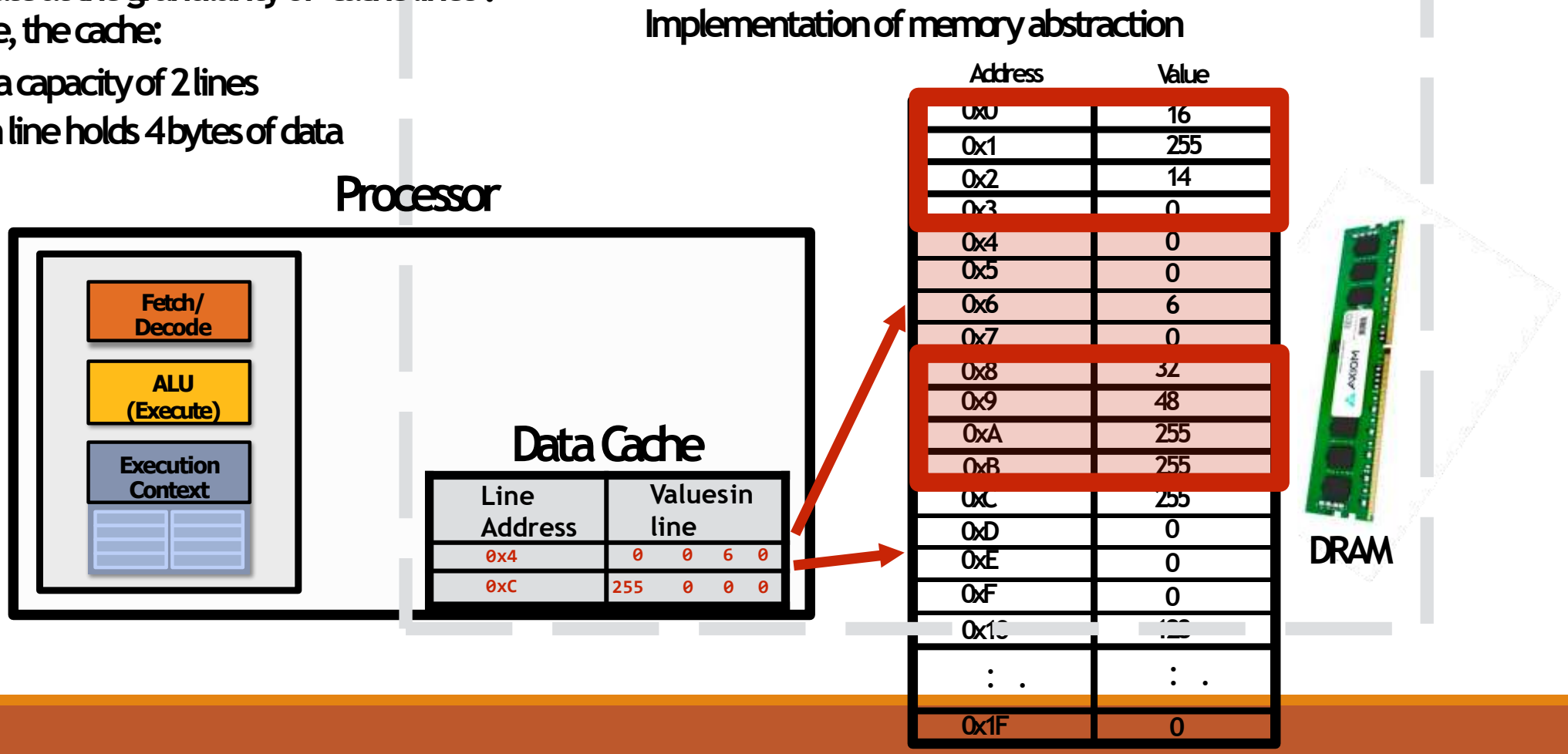
Caches/C-languge intro /Flynn Taxonomy /Multi threading

By: Muhammad Zain Uddin

email: zuddin@iba.edu.pk

What are caches?

- A cache is a hardware implementation detail that does not impact the output of a program, only its performance
- Cache is on-chip storage that maintains a copy of a subset of the values in memory
- If an address is stored "in the cache" the processor can load/store to this address more quickly than if the data resides only in DRAM
- Caches operate at the granularity of "cache lines".
  In the figure, the cache:
  - Has a capacity of 2 lines
  - Each line holds 4 bytes of data



Cache example 1

Array of 16 bytes in memory

	Address	Value
Line 0x0	0x0	16
	0x1	255
	0x2	14
	0x3	0
Line 0x4	0x4	0
	0x5	0
	0x6	6
	0x7	0
Line 0x8	0x8	32
	0x9	43
Line 0xC	0xA	25
	0xB	25
	0xC	25

Assume:

Total cache capacity of 8 bytes

Cache with 4-byte cache lines (So 2 lines fit in cache)

Least recently used (LRU) replacement policy

time

Address accessed	Cache action	Cache state (after load is complete)
0x0	"cold miss", load	0x0
0x0 0x1	hit	0x0
0x2	hit	0x0
0x3	hit	0x0
0x2	hit	0x0
0x1	hit	0x0
0x4	"cold miss", load	0x0 0x4
0x4 0x1	hit	0x0 0x4

There are two forms of "data locality" in this sequence:

Spatial locality: loading data in a cache line "preloads" the data needed for subsequent accesses to <u>different addresses</u> in the same line, leading to cache hits

Temporal locality: repeated accesses to the <u>same address</u> result in hits.

Cache example 2

Array of 16 bytes in memory

Address	Value
0x0	16
0x1	255
0x2	14
0x3	0
0x4	0
0x5	0
0x6	6
0x7	0
0x8	32
0x9	48
0xA	255
0xB	255
0xC	255
0xD	0

Line 0x0

Line 0x4

Line 0x8

Line 0xC

Assume:

Total cache capacity of 8 bytes

Cache with 4-byte cache lines (So 2 lines fit in cache)

Least recently used (LRU) replacement policy

time

Address accessed	Cache action	Cache state (after load is complete)	
0x0	"cold miss", load	0x0	
0x0, 0x1	hit	0x0	
0x2	hit	0x0	
0x3	hit	0x0	
0x4	"cold miss", load	0x0	0x4
0x4, 0x5	hit	0x0	0x4
0x6	hit	0x0	0x4
0x7	hit	0x0	0x4
0x8	"cold miss", load 0x8 (evict	0x8	0x4
0x0, 0x9	hit	0x8	0x4
0xA	hit	0x8	0x4
0xB	hit	0x8	0x4
0xC	"cold miss", load 0xC (evict	0x8	0xC
0x4, 0xD	hit	0x8	0xC
0xE	hit	0x8	0xC
0xF	hit	0x8	0xC
0x0	"capacity miss", load 0x0 (evict 0x8)	0x0	0xC

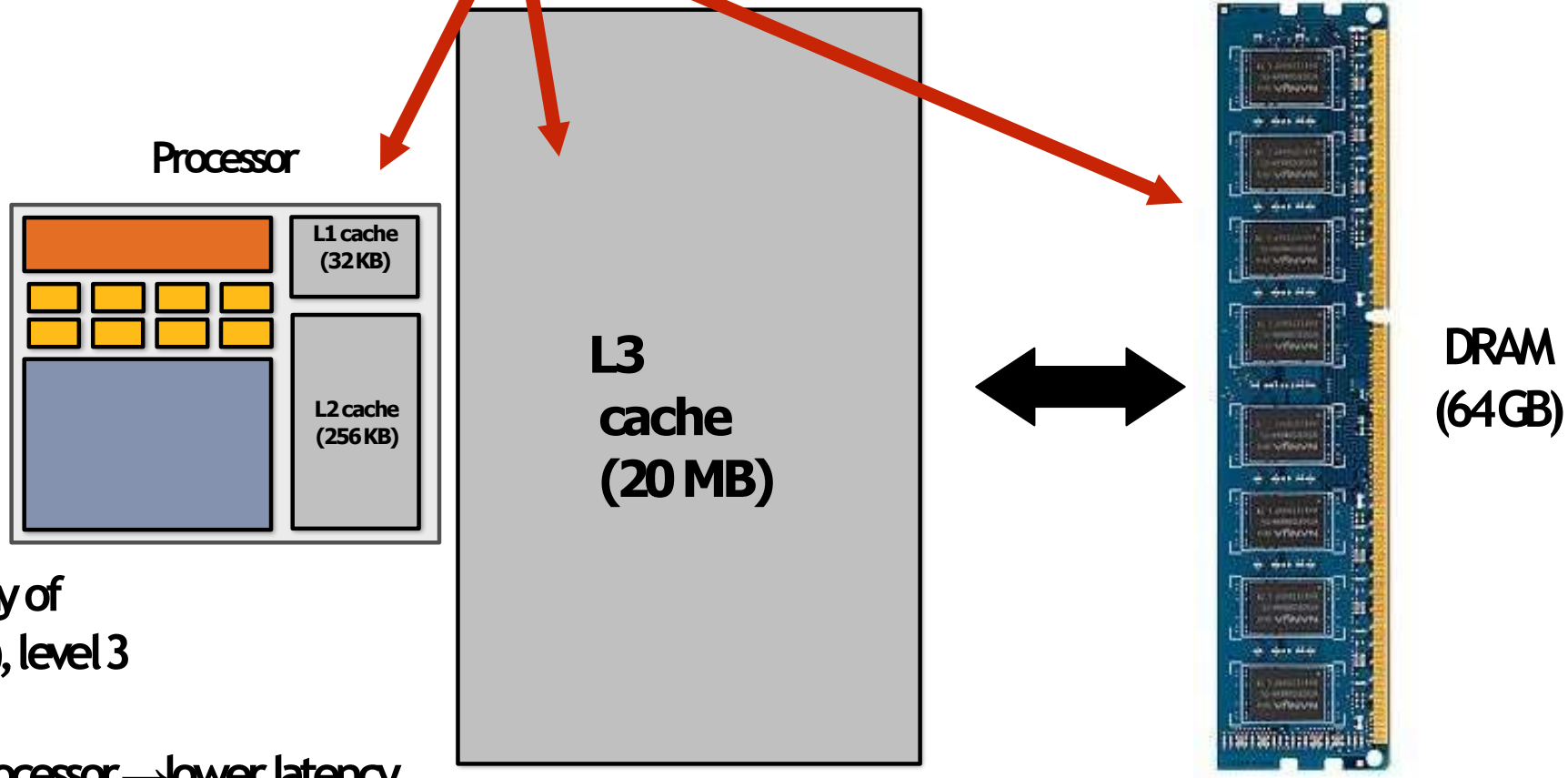
Caches reduce length of stalls (reduce memory access latency)

- **Processors run efficiently when they access data that is resident in caches**
- **Caches reduce memory access latency when processors accesses data that they have recently accessed! ***

*** Caches also provide high bandwidth data transfer**

The implementation of the linear memory address space abstraction on a modern computer is complex

The instruction "load the value stored at address X into register R0" might involve a complex sequence of operations by multiple data caches and access to DRAM



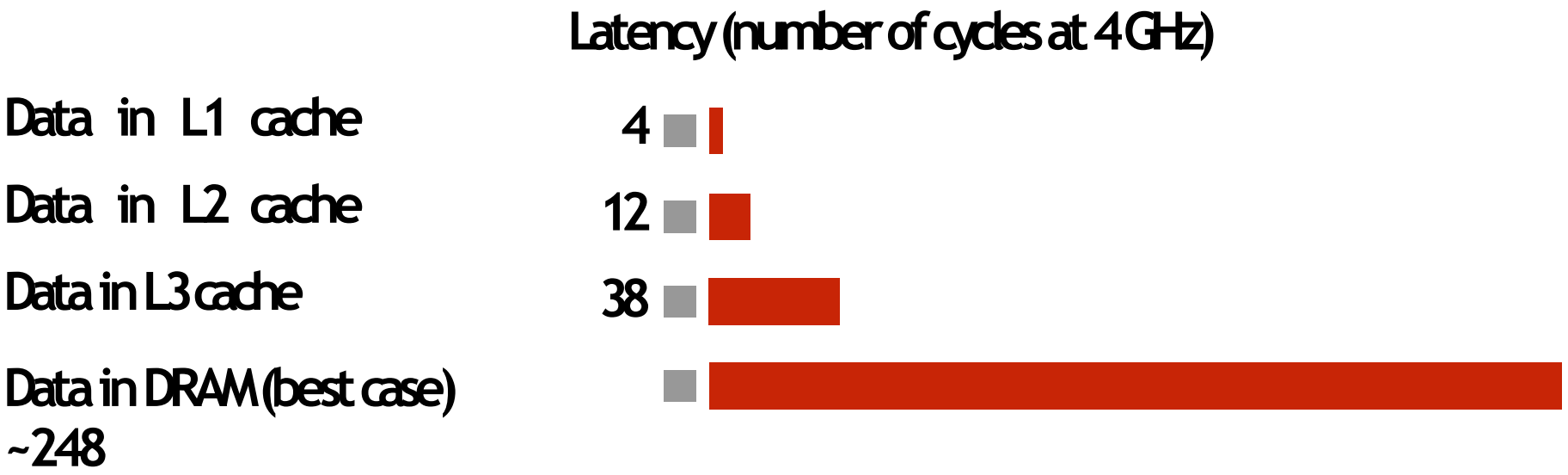
Common organization: hierarchy of caches: Level 1 (L1), level 2 (L2), level 3 (L3)

Smaller capacity caches near processor → lower latency

Larger capacity caches farther away → larger latency

Data access times

(Kaby Lake CPU)



Now lets get back to basic understanding of C

```
int main(int argc, char** argv)
                {
 int x = 1;

  for (int i=0; i<10; i++)
         { x = x + x;
              }

 printf("%d\n", x);

 return 0;
}
```

But what exactly does this code does,

```
int main(int argc, char** argv) {

    int x = 1;

    for (int i=0; i<10; i++) {
        x = x + x;
    }

    printf("%d\n", x);

    return 0;
}
```

?

?

?

?

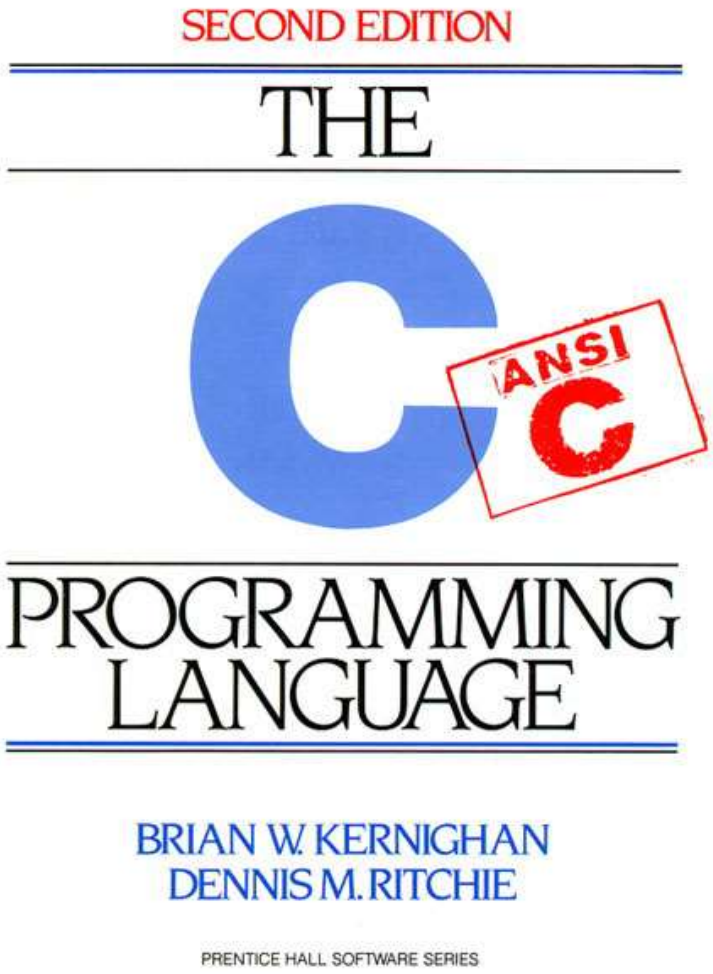
Introduction to C (1/2)

Kernighan and Ritchie

- *C is not a "very high-level" language, nor a "big" one, and is not specialized to any particular area of application. But its absence of restrictions and its generality make it more convenient and effective for many tasks than supposedly more powerful languages.*

Enabled first operating system not written in assembly language!

- UNIX - A portable OS!



Introduction to C (2/2)

Why C?

- We can write programs that allow us to exploit underlying features of the architecture
  - memory management, special instructions, parallelism

C and derivatives (C++/Obj-C/C#) still one of the most popular programming languages after >40 years!

If you are starting a new project where performance matters use either Go or Rust

- Rust, “C-but-safe”: By the time your C is (theoretically) correct w/all necessary checks it should be no faster than Rust
- Go, “Concurrency”: Practical concurrent programming to take advantage of modern multi-core microprocessors

Compile
v.
Interpret

Compilation: Overview

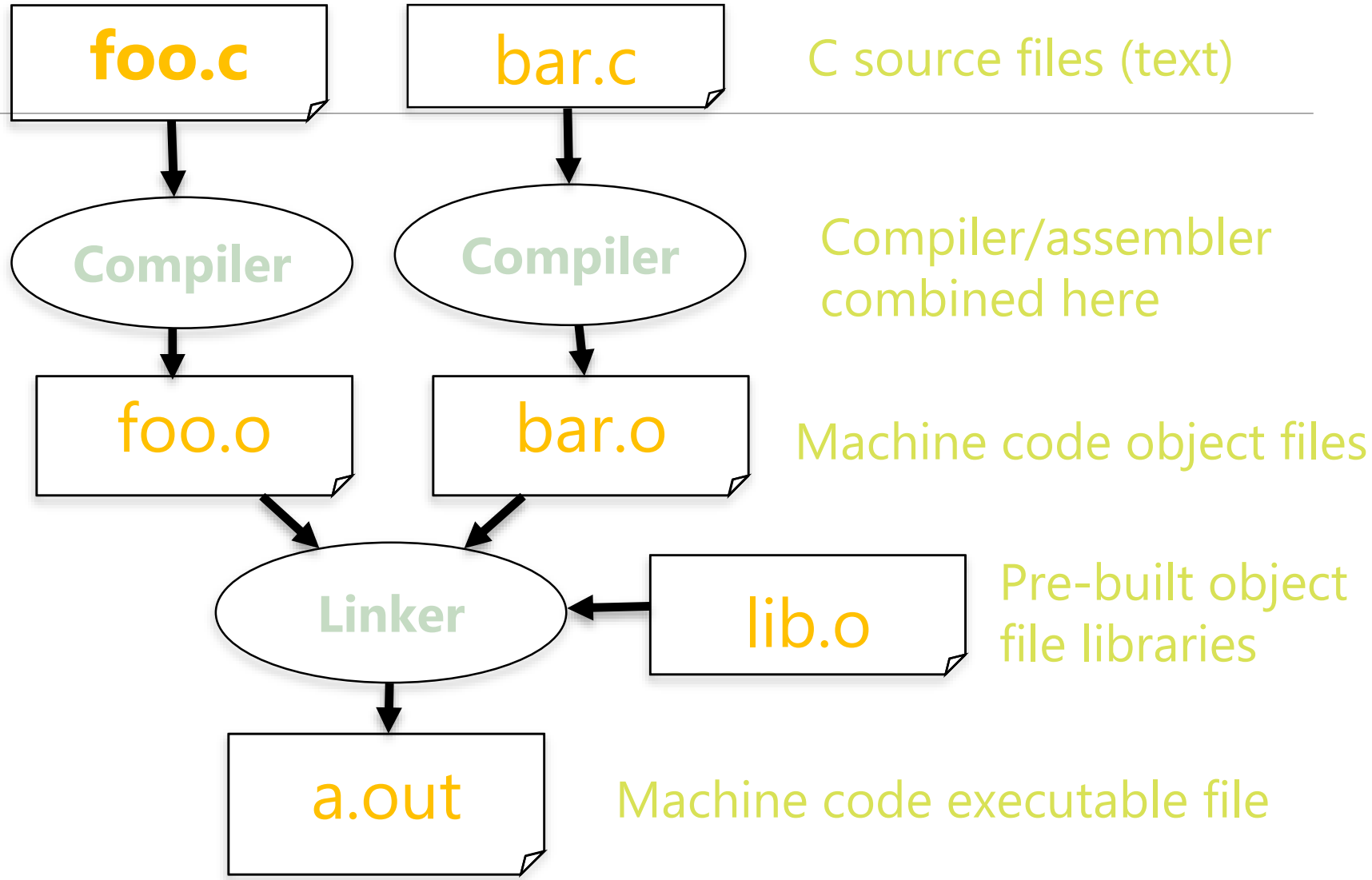
C compilers map C programs directly into architecture-specific machine code (string of 1s and 0s)

- Unlike Java, which converts to architecture-independent bytecode that may then be compiled by a just-in-time compiler (JIT)
- Unlike Python environments, which converts to a byte code at runtime
  - These differ mainly in exactly when your program is converted to low-level machine instructions ("levels of interpretation")

For C, generally a two part process of compiling .c files to .o files, then linking the .o files into executables;

- Assembling is also done (but is hidden, i.e., done automatically, by default); we'll talk about that later

C Compilation Simplified Overview (more later)



C vs Java

C vs. Java (1/3)

	C	Java
Type of Language	Function Oriented	Object Oriented
Programming Unit	Function	Class = Abstract Data Type
Compilation	`gcc hello.c` creates machine language code	`javac Hello.java` creates Java virtual machine language bytecode
Execution	`a.out` loads and executes program	`java Hello` interprets bytecodes

hello, world	`#include <stdio.h>`
	`int main(void)`
	`{`
	`  printf("Hi\n");`
	`  return 0;`
	`}`
		`public class HelloWorld {`
		` public static void`
		`main(String[] args) {`
		`  System.out.println("Hi");`
		`} }`

Storage	Manual (`malloc`, `free`)	New allocates & initializes, Automatic (garbage collection) frees

From http://www.cs.princeton.edu/introcs/faq/c2java.html

C vs. Java (2/3)

	C	Java
Comments (C99 same as Java)	`/* … */`	`/* … */` or `//` … end of line
Constants	`#define`, `const`	`final`
Preprocessor	Yes	No
Variable declaration (C99 same as Java)	At beginning of a block	Before you use it
Variable naming conventions	`sum_of_squares`	`sumOfSquares`
Accessing a library	`#include <stdio.h>`	`import java.io.File;`

From http://www.cs.princeton.edu/introcs/faq/c2java.html

C vs. Java (3/3) …operators nearly identical

- arithmetic: `+, -, *, /, %`
- assignment: `=`
- augmented assignment: `+=, -=, *=, /=, %=, &=, |=, ^=, <<=, >>=`
- bitwise logic: `~, &, |, ^`
- bitwise shifts: `<<, >>`
- boolean logic: `!, &&, ||`
- equality testing: `==, !=`
- subexpression grouping: `()`
- order relations: `<, <=, >, >=`
- increment and decrement: `++` and `--`
- member selection: `., ->`
  - Slightly different than Java because there are both structures and pointers to structures, more later
- conditional evaluation: `? :`

C Syntax: main

To get the **`main`** function to accept arguments, use this:

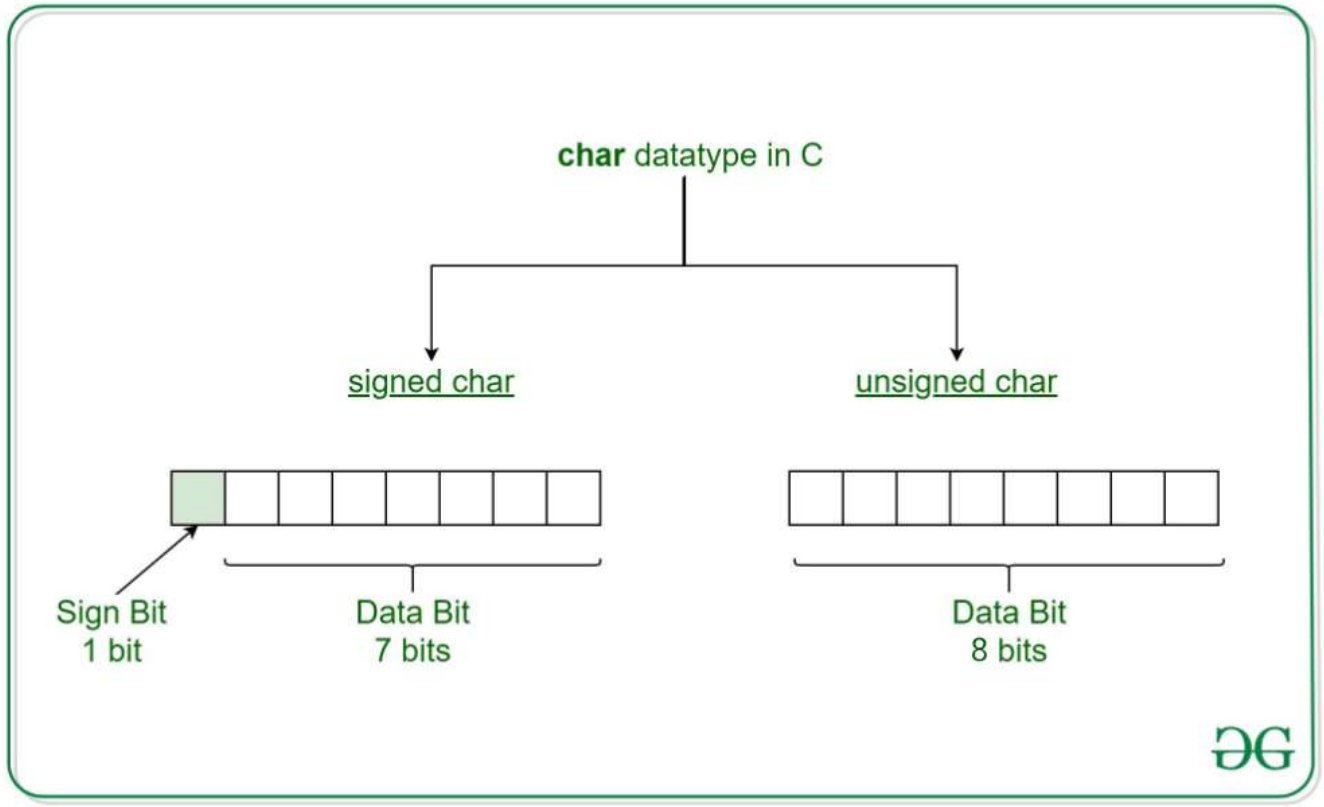
- **`int main (int argc, char *argv[])`**

What does this mean?

- **`argc`** will contain the number of strings on the command line (the executable counts as one, plus one for each argument). Here **`argc`** is 2:
  - **`unix% sort myFile`**
- **`argv`** is a pointer to an array containing the arguments as strings (more on pointers later).

Type	Storage size	Value range
char	1 byte	-128 to 127 or 0 to 255
unsigned char	1 byte	0 to 255
signed char	1 byte	-128 to 127
int	2 or 4 bytes	-32,768 to 32,767 or -2,147,483,648 to 2,147,483,647
unsigned int	2 or 4 bytes	0 to 65,535 or 0 to 4,294,967,295
short	2 bytes	-32,768 to 32,767
unsigned short	2 bytes	0 to 65,535
long	8 bytes or (4bytes for 32 bit OS)	-9223372036854775808 to 9223372036854775807
unsigned long	8 bytes	0 to 18446744073709551615

Char


char datatype in C
signed char
unsigned char
Sign Bit
1 bit
Data Bit
7 bits
Data Bit
8 bits

int main(int argc, char** argv)

argc stands for "argument count," and argv stands for "argument vector." These parameters are commonly used in C and C++ programs to handle command-line arguments when the program is executed from the command line.

- argc: It is an integer that represents the number of command-line arguments passed to the program, including the program name itself. The value of argc is at least 1, as it includes the program name.

- argv: It is an array of strings (char**), where each element is a pointer to a null-terminated string representing a command-line argument. argv[0] is the program name, and subsequent elements contain the additional arguments.

Another example

```
#include <stdio.h>

int main(int argc, char** argv) {
    // Print the program name (first argument)
    printf("Program Name: %s\n", argv[0]);

    // Print the count of command-line arguments
    printf("Argument Count: %d\n", argc);

    // Print each command-line argument
    printf("Arguments:\n");
    for (int i = 0; i < argc; ++i) {
        printf("argv[%d]: %s\n", i, argv[i]);
    }

    return 0;
}
```

Command after saving the file:
./myprogram arg1 arg2 arg3

Output will be

Program Name: ./myprogram

Argument Count: 4

Arguments:

argv[0]: ./myprogram

argv[1]: arg1

argv[2]: arg2

argv[3]: arg3

But why did we use **“char** argv”**

This is alternative of “**(char argv[][MAX_LENGTH])” so,**

argv as an Array of Strings,

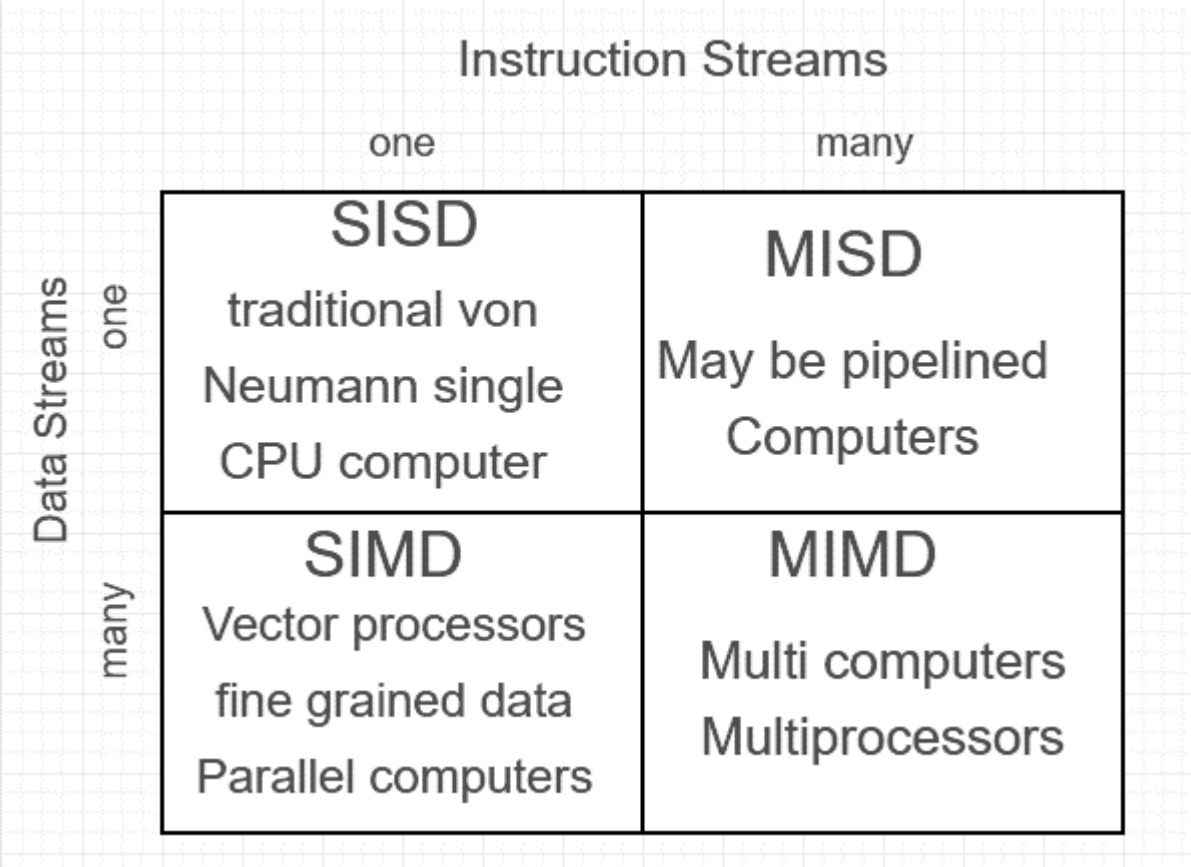
char for Strings,**

argv as an Array of Pointers,

Advantage of pointer

- Using char** for argv allows for flexibility. Each argv[i] can point to a different string of varying lengths. This is in contrast to using a two-dimensional array (char argv[][MAX_LENGTH]), where all strings would need to have the same fixed size.
- By using pointers, only the addresses of strings are stored in memory, not the actual string contents. This is memory-efficient, especially when dealing with a large number of arguments or long strings.

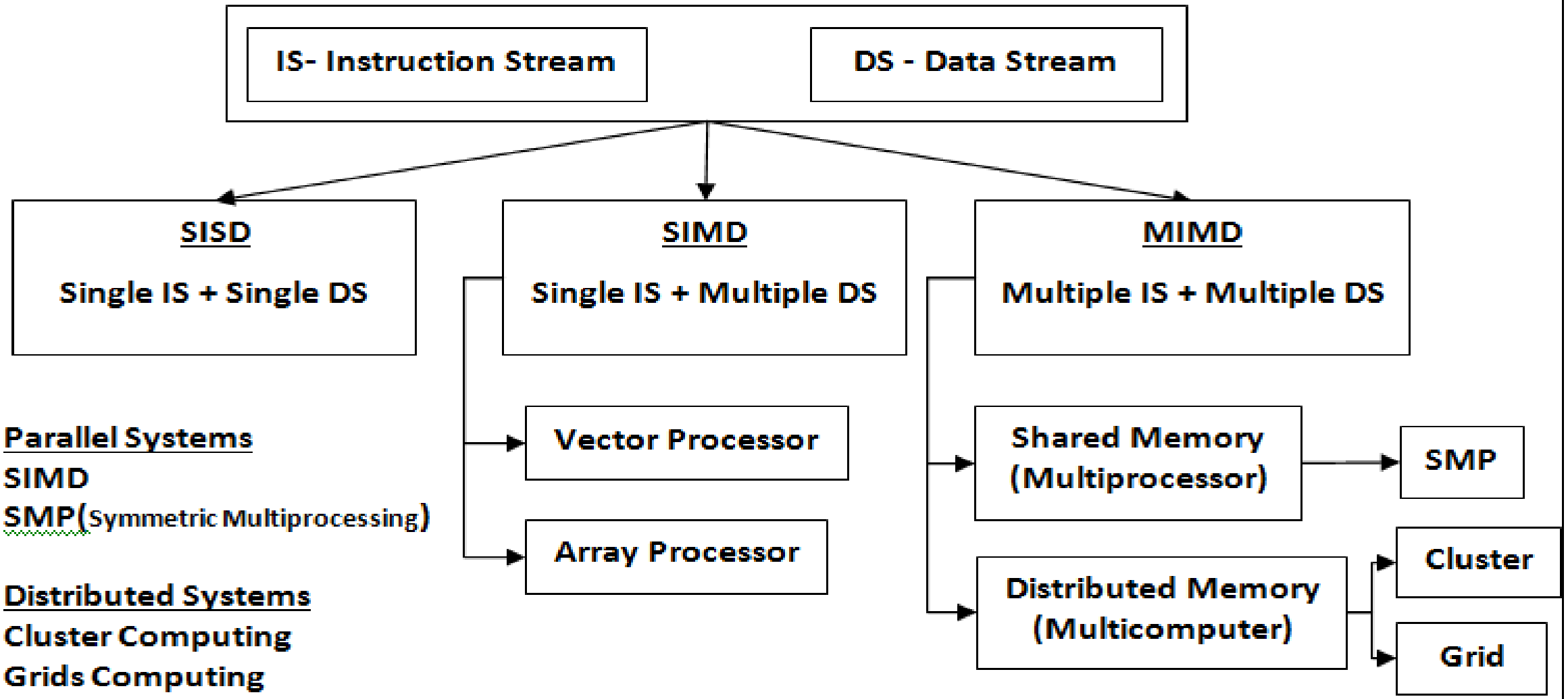
Understanding Different Flavors of Parallelism



- SISD:
  - Single instruction stream, single data stream.
  - Traditional single-core CPUs fall under this category.
- SIMD:
  - Single instruction stream, multiple data streams.
  - Vector processors and GPUs excel at this type of parallelism.
- MIMD:
  - Multiple instruction streams, multiple data streams.
  - Most parallel and distributed systems belong to this category.

Source: https://media.geeksforgeeks.org/wp-content/uploads/cao.png

Flynn's Taxonomy

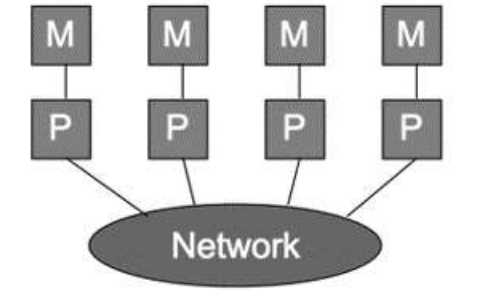


Parallel Systems
SIMD
SMP(Symmetric Multiprocessing)

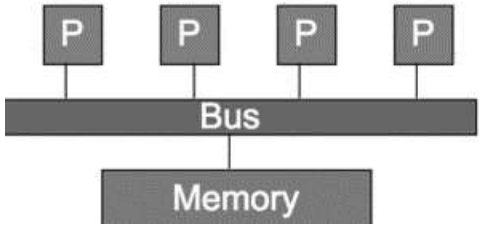
Distributed Systems
Cluster Computing
Grids Computing

Dive Deeper into MIMD Architectures

- Focus: Shared memory vs. distributed memory MIMD systems.
- Shared memory: Processors share a common memory space for direct data access.
- Distributed memory: Each processor has its own local memory, requiring message passing for communication.



- **Distributed memory**
  - Each processor has its own local memory
  - Message-passing is used to exchange data between processors



- **Shared memory**
  - Single address space
  - All processes have access to the pool of shared memory

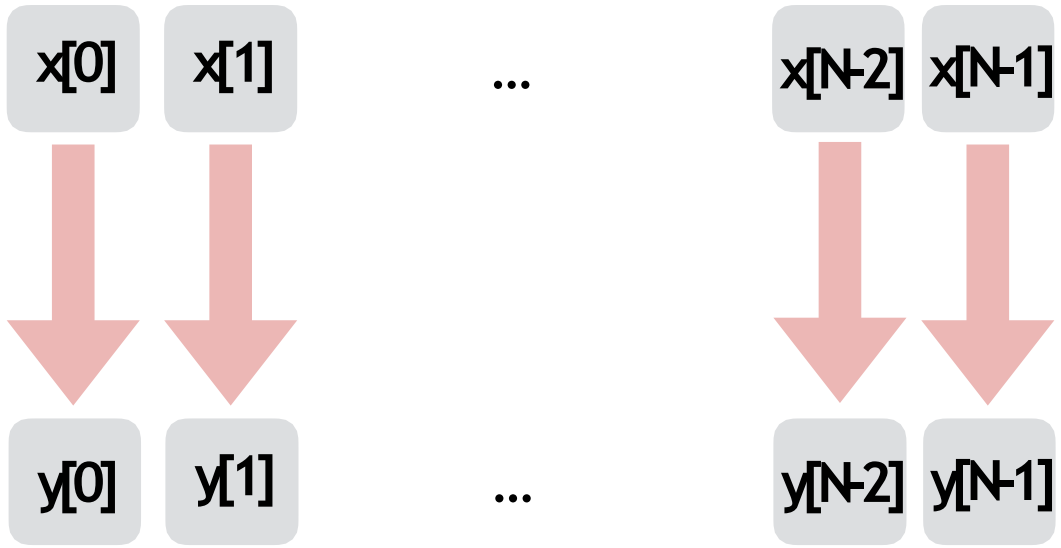
example program

```
void sinx(int N, int terms, float* x, float* y)
{
   for (int i=0; i<N; i++)
   {
      float value = x[i];
      float numer = x[i] * x[i] * x[i];
      int denom = 6;  // 3!
      int sign = -1;

      for (int j=1; j<=terms; j++)
      {
         value  +=  sign * numer / denom;
         numer  *=  x[i] * x[i];
         denom  *=  (2*j+2) * (2*j+3);
         sign *= -1;
      }

      y[i] = value;
   }
}
```

Compute $\sin(x)$ **using Taylor expansion:**

$$\sin(x) = x - x^3/3! + x^5/5! - x^7/7! + \ldots$$

for each element of an array of N floating-point numbers

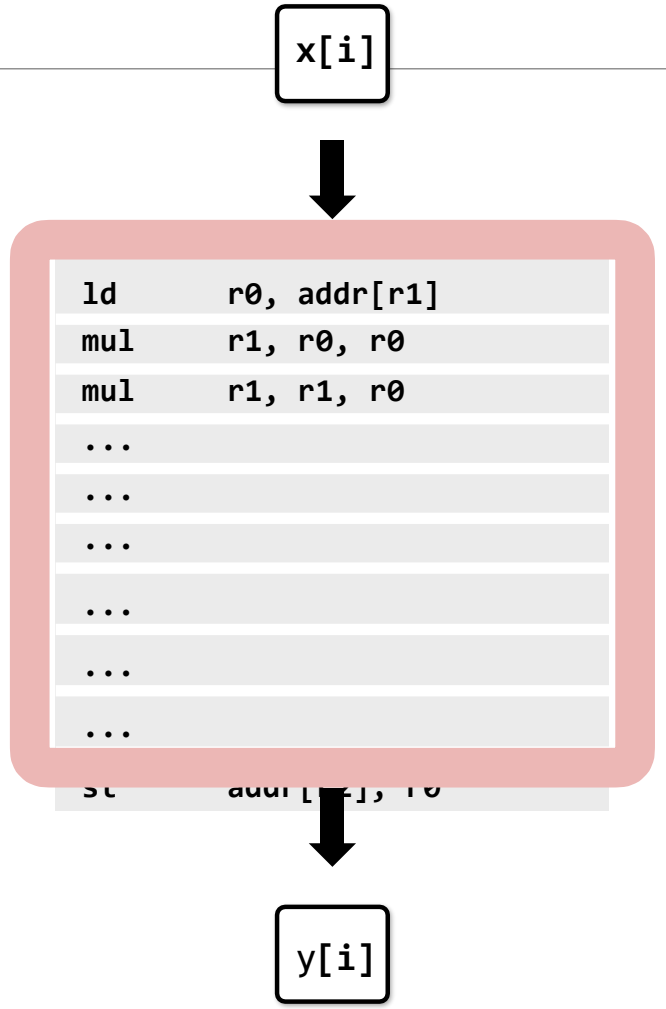


Compile program

```
void sinx(int N,   int terms, float* x, float* y)
{
     for (int      i<N;  i++)
     {float value = x[i];
      float numer = x[i] * x[i] * x[i];
      int denom = 6;  // 3!
      int sign = -1;

      for   (int   j=1;   j<=terms;    j++)
      {
           value  +=  sign * numer / denom;
           numer  *=  x[i] * x[i];
           denom  *=  (2*j+2) * (2*j+3);
             sign *= -1;
          }

       y[i] = value;
   }
}
```

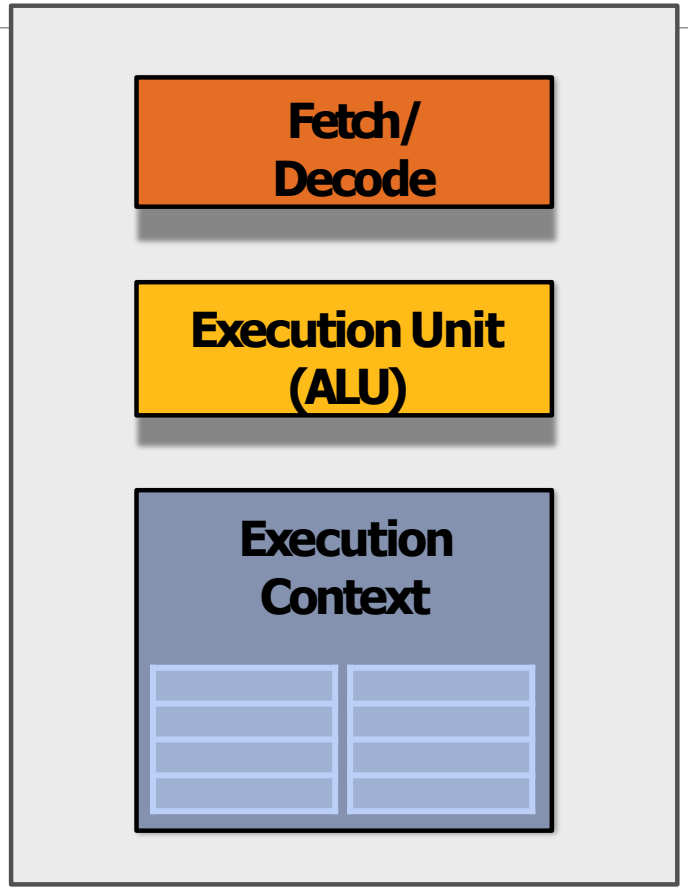
compiler

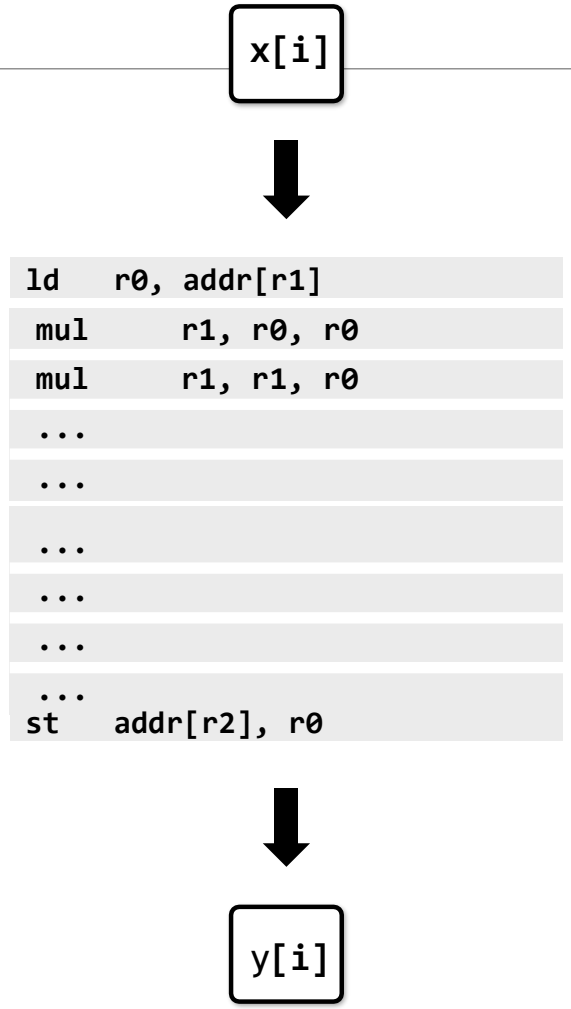
Compiled instruction stream
(scalar instructions)



Execute program

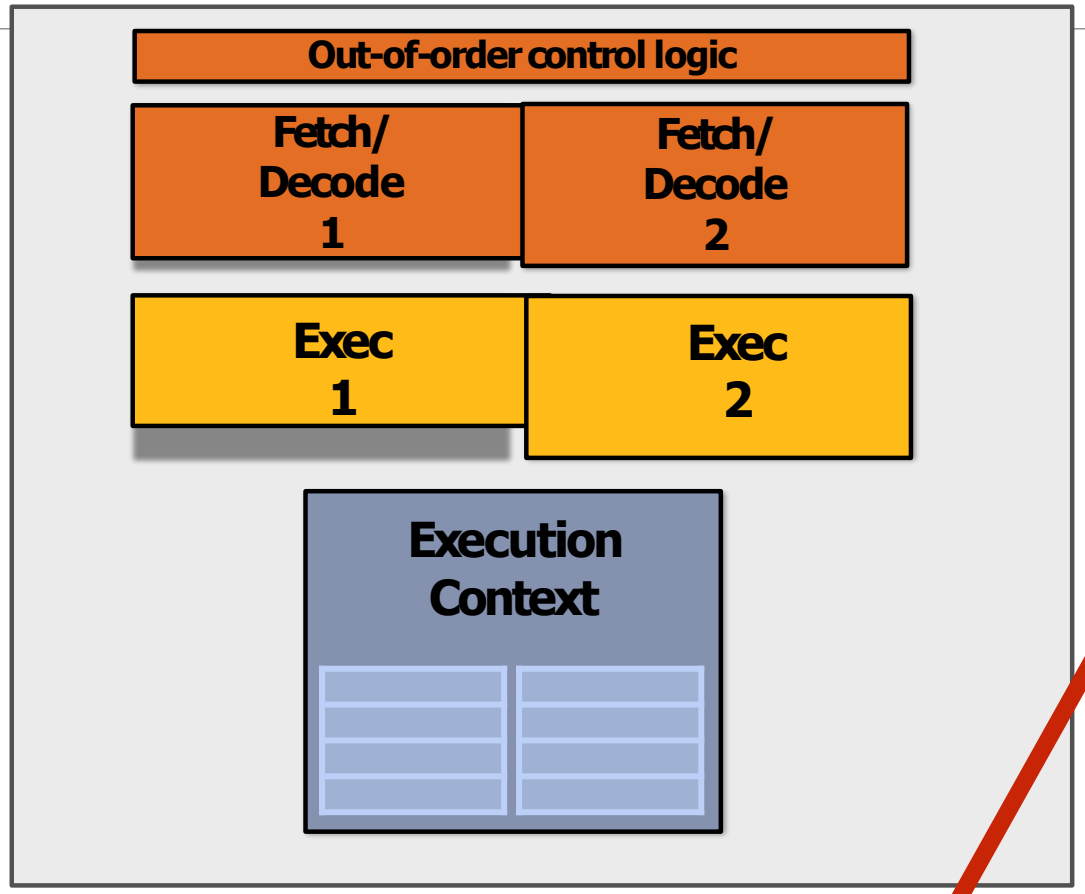
My very simple processor: executes one instruction per clock

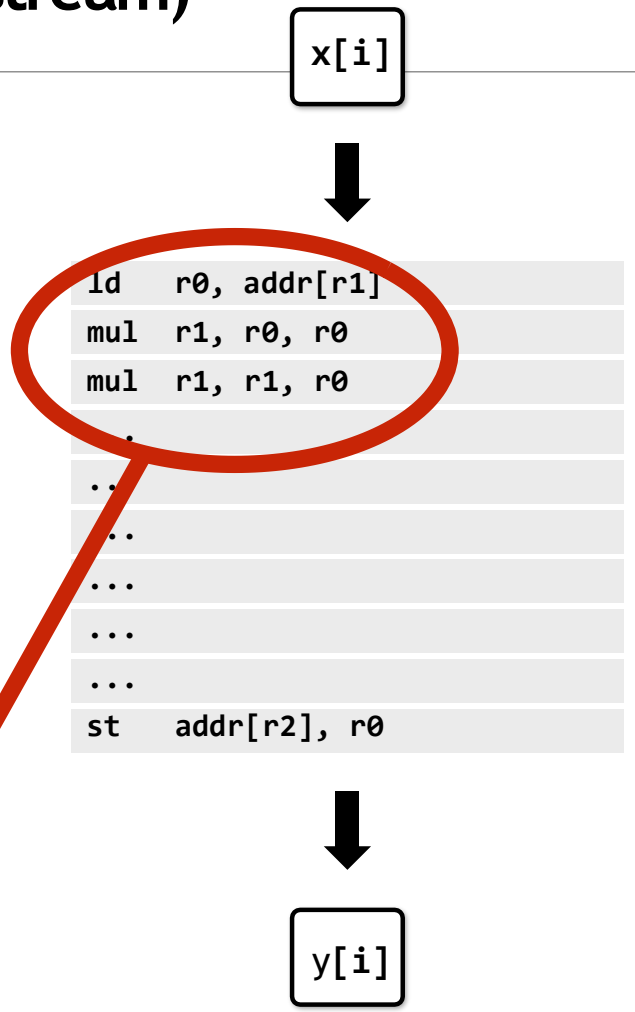




Superscalar processor

The processor shown here can decode and execute two instructions per clock (if independent instructions exist in an instruction stream)

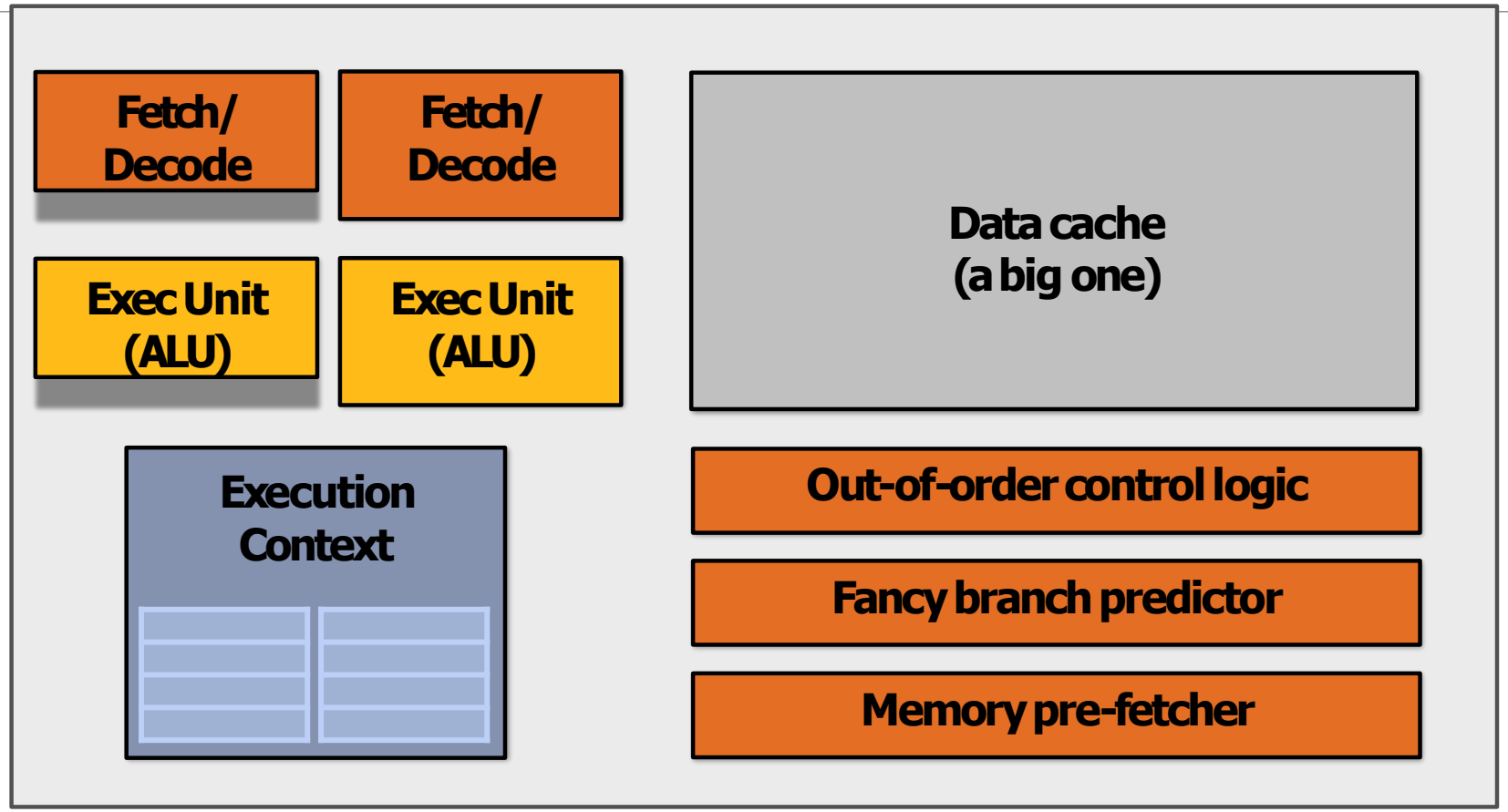




Note: No ILP exists in this region of the program

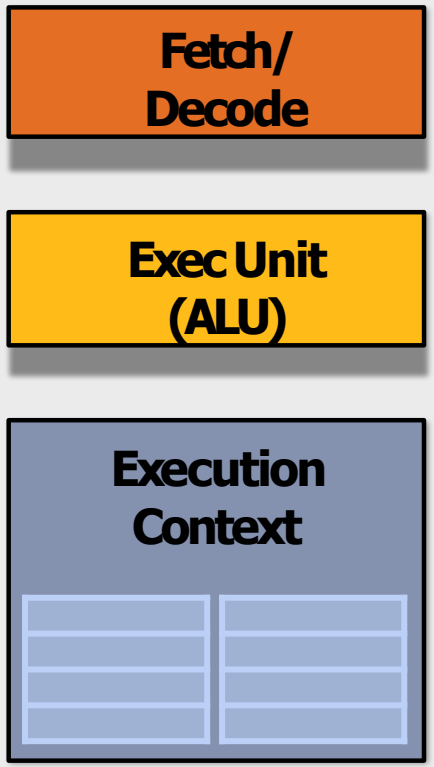
Pre multi-core era processor

Majority of chip transistors used to perform operations that help make a single instruction stream run fast



More transistors = larger cache, smarter out-of-order logic, smarter branch predictor, etc.

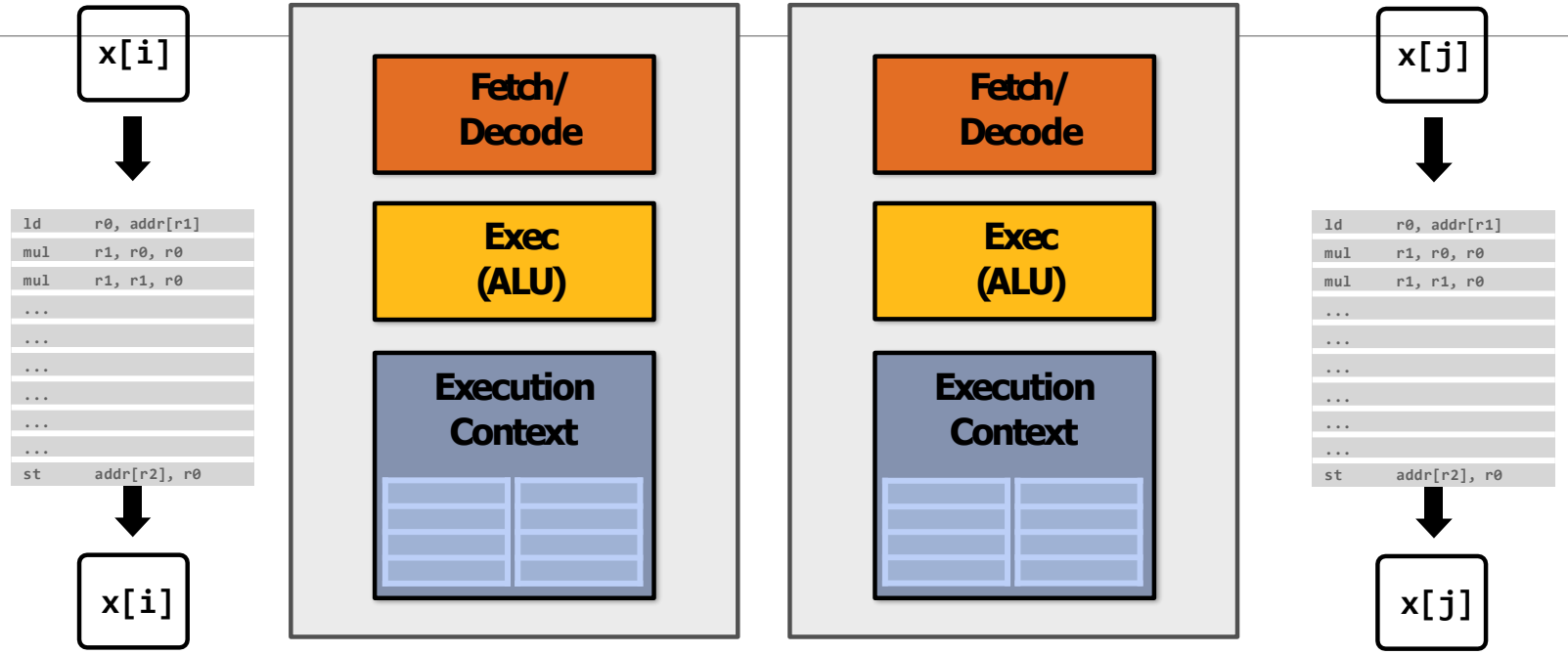
Multi-core era processor



Idea #1:

Rather than use transistors to increase sophistication of processor logic that accelerates a single instruction stream
(e.g., out-of-order and speculative operations)
Use increasing transistor count to add more cores to the processor

Two cores: compute two elements in parallel



Simpler cores: each core may be slower at running a single instruction stream than our original "fancy" core (e.g., 25% slower)

But there are now two cores: 2 × 0.75 = 1.5 (potential for speedup!)

But our program expresses no parallelism

```
void sinx(int N, int terms, float* x, float*
y)
{
   for (int i=0; i<N; i++)
   {
      float value  = x[i];
      float numer  = x[i] * x[i]  * x[i];
      int denom = 6;  // 3!
      int sign = -1;

    for   (int   j=1;   j<=terms;      j++)
    {
         value    +=  sign * numer / denom;
         numer    *=  x[i] * x[i];
         denom    *=  (2*j+2) * (2*j+3);
          sign *= -1;
      }

      y[i] = value;
   }
}
```

This C program will compile to an instruction stream that runs as one thread on one processor core.

If each of the simpler processor cores was 25% slower than the original single complicated one, our program now runs 25% slower than before.

! 😭

Example: expressing parallelism using C++ threads

```
typedef struct {
   int N;
   int terms;
   float* x;
   float* y;
} my_args;

void my_thread_func(my_args* args)
{
   sinx(args->N, args->terms, args->x, args->y); // do work
}
```

```
void parallel_sinx(int N, int terms, float* x, float* y)
{
   std::thread my_thread;
   my_args args;

   args.N = N/2;
   args.terms = terms;
   args.x = x;
   args.y = y;

   my_thread = std::thread(my_thread_func, &args); // launch thread
   sinx(N - args.N, terms, x + args.N, y + args.N); // do work on main
   thread my_thread.join();       // wait for thread to complete
}
```

```
void sinx(int N, int terms, float* x, float*
y)
{
   for (int i=0; i<N; i++)
   {
      float value = x[i];
      float numer = x[i] * x[i] *
      x[i]; int denom = 6;        // 3!
      int sign = -1;

      for (int j=1; j<=terms; j++)
      {
         value += sign * numer /
         denom numer *= x[i] * x[i];
         denom *= (2*j+2) *
         (2*j+3); sign *= -1;
      }

   }  y[i] = value;
}
```

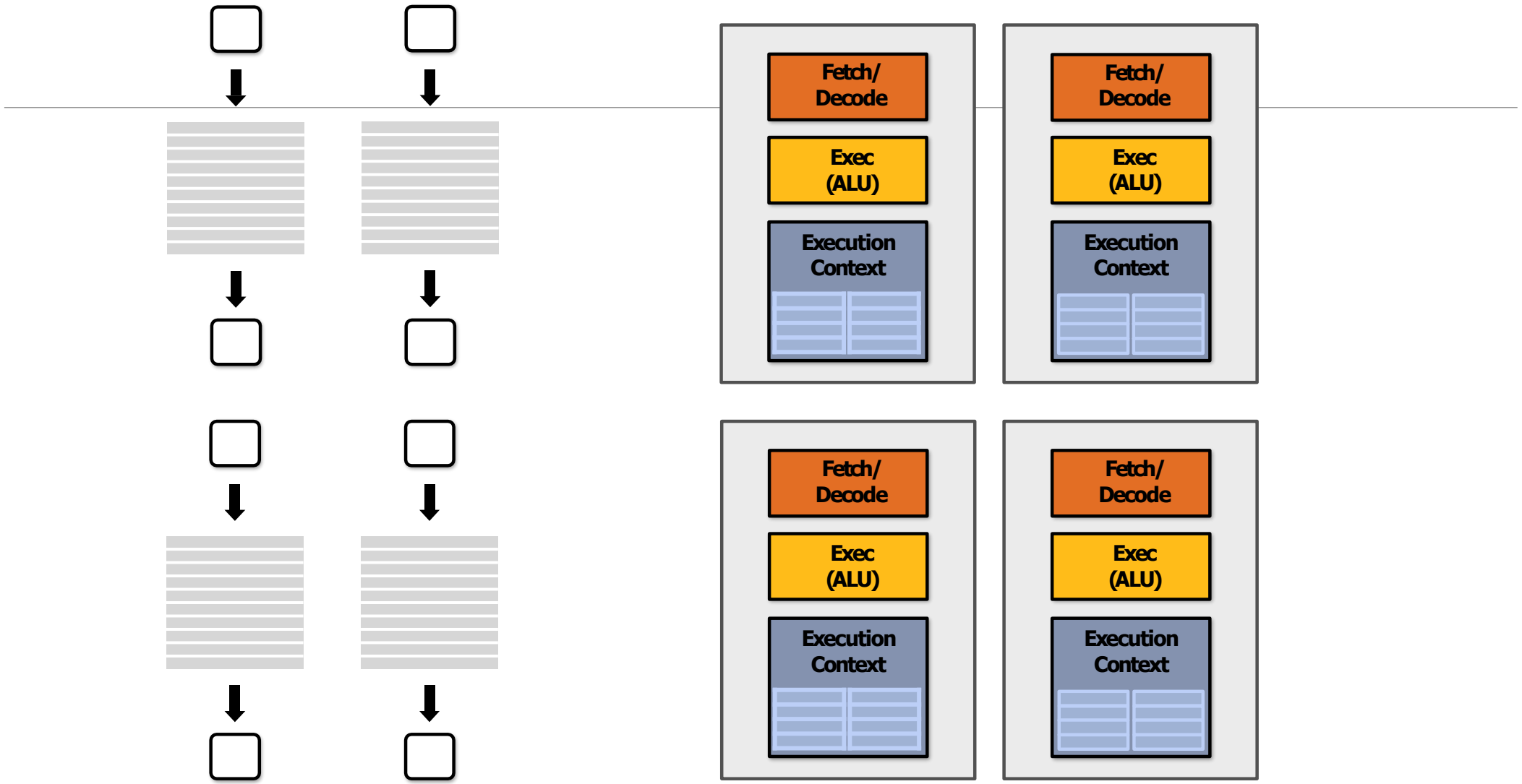
Data-parallel expression (in Kayvon's fictitious programming language with a "forall" construct)

```
void sinx(int N, int terms, float* x, float* y)
{
   // declares that loop iterations are independent
   forall (int i from 0 to N)
   {
      float value = x[i];
      float numer = x[i] * x[i] * x[i];
      int denom = 6;  // 3!
      int sign = -1;

      for (int j=1; j<=terms; j++)
      {
         value += sign * numer / denom;
         numer *= x[i] * x[i];
         denom *= (2*j+2) * (2*j+3);
         sign *= -1;
      }

      y[i] = value;
   }
}
```

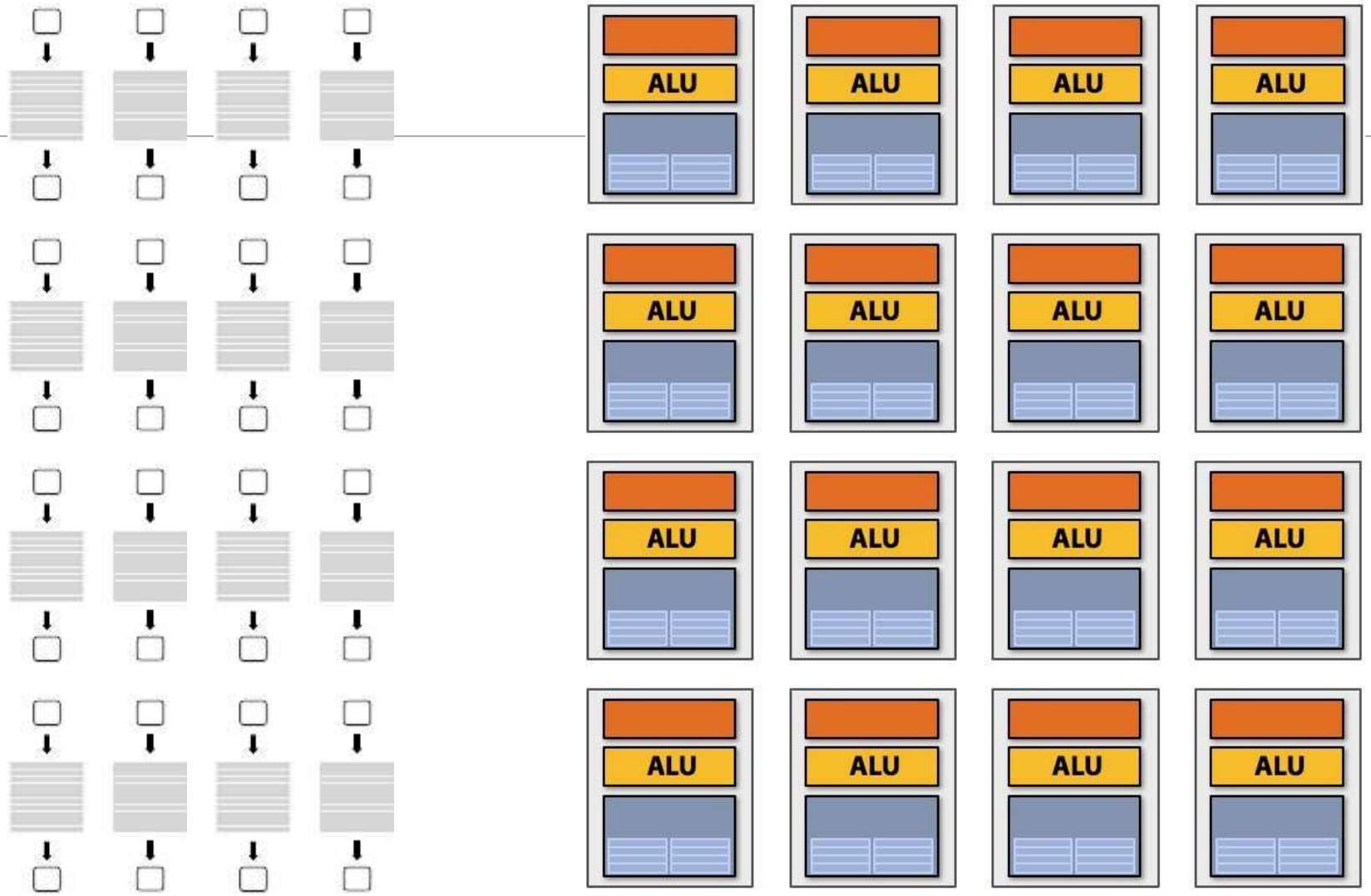
In this code, loop iterations are declared by the programmer to be independent (see the 'forall')

With this information, you could imagine how a compiler might automatically generate threaded code for you.

Four cores: compute four elements in parallel



Sixteen cores: compute sixteen elements in parallel



Sixteen cores, sixteen simultaneous instruction streams

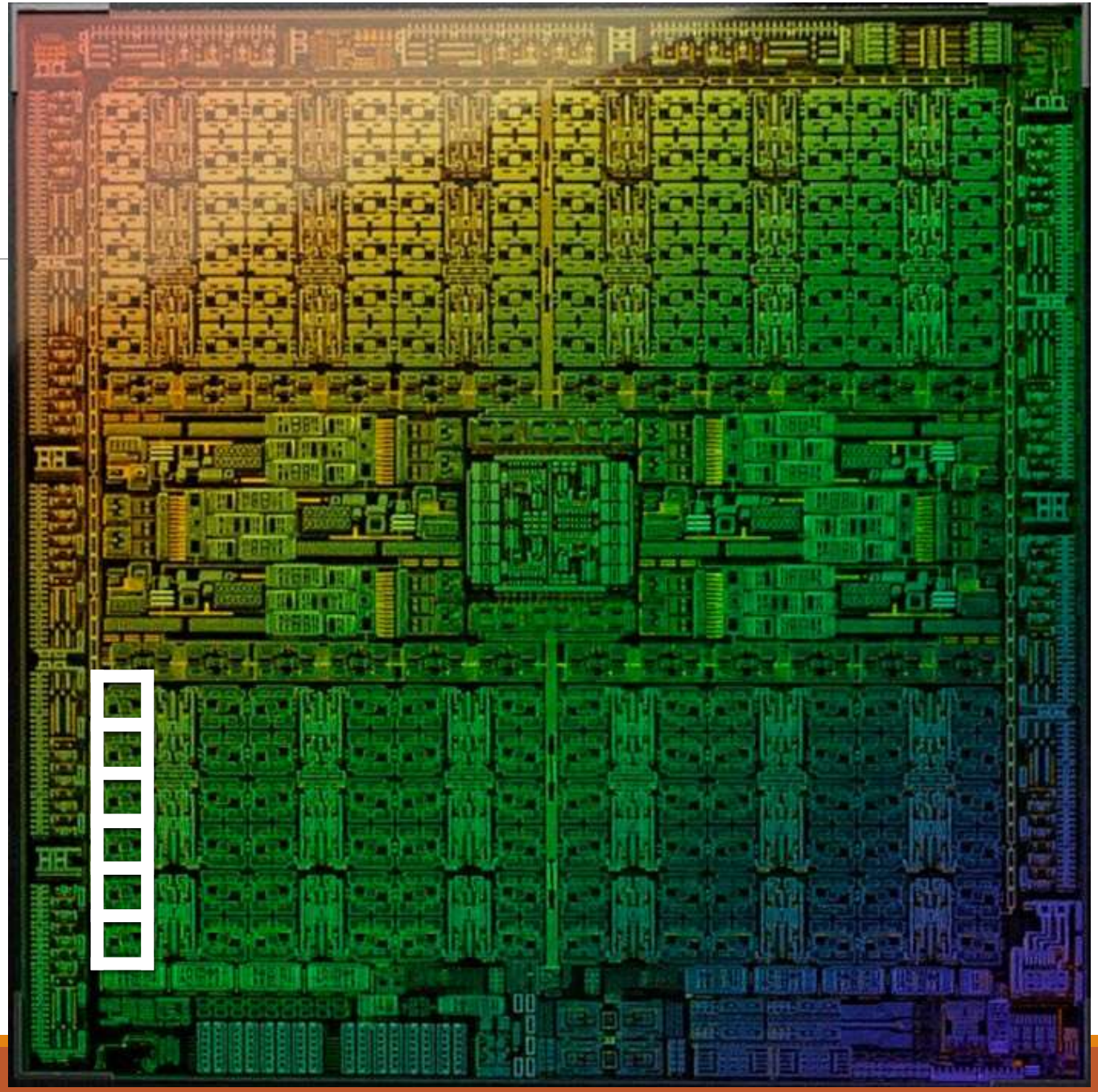
Example: multi-core CPU

Intel “Comet Lake” 10th Generation Core i9 10-core CPU (2020)

Multi-core GPU

GeForce RTX 4090 (2022)

144 processing blocks (called SMs)



Apple A15 Bionic

Two "big cores" + four "small" cores

2 "big" CPU cores

4 "small" CPU cores

Image Credit: TechInsights Inc.

Data-parallel expression

(in Kayvon's fictitious programming language with a "forall" construct)

```
void sinx(int N, int terms, float* x, float* result)
{
   // declares that loop iterations are independent
   forall (int i from 0 to N)
   {
      float value = x[i];
      float numer = x[i] * x[i] * x[i];
      int denom = 6;  // 3!
      int sign = -1;
   for   (int   j=1; j<=terms;       j++)
   {
        value   +=  sign * numer / denom;
        numer   *=  x[i] * x[i];
        denom   *=  (2*j+2) * (2*j+3);
         sign *= -1;
      }

      result[i] = value;
   }
}
```

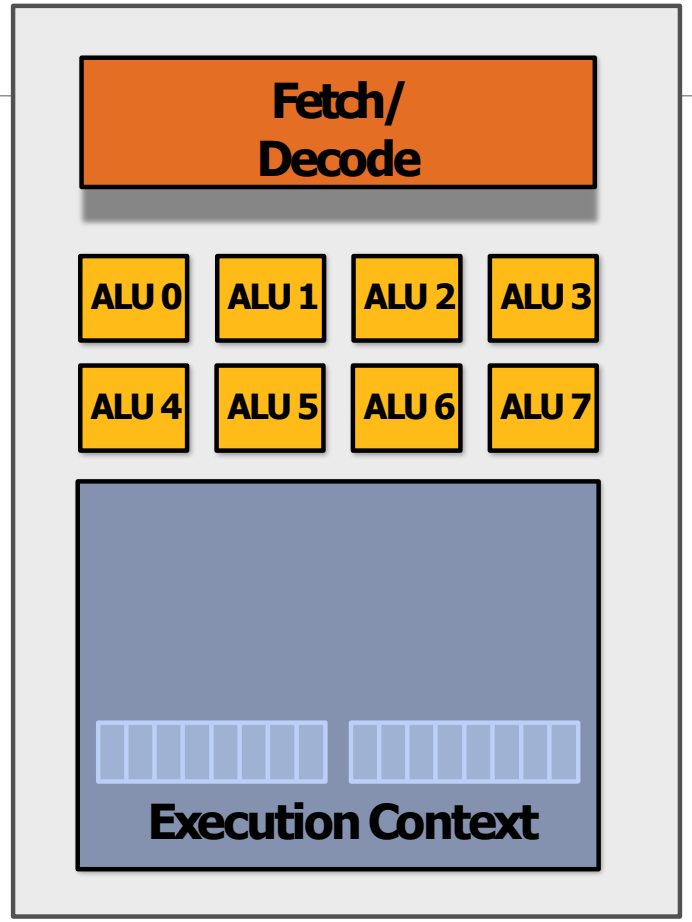
Another interesting property of this code:

Parallelism is across iterations of the loop.

All the iterations of the loop carry out the exact same sequence of instructions (defined by the loop body), but on different input data given by x[i]

(the loop body computes sine(x[i]))

Add execution units (ALUs) to increase compute capability



Idea #2:
Amortize cost/complexity of managing an instruction stream across many ALUs

SIMD processing

Single instruction, multiple data

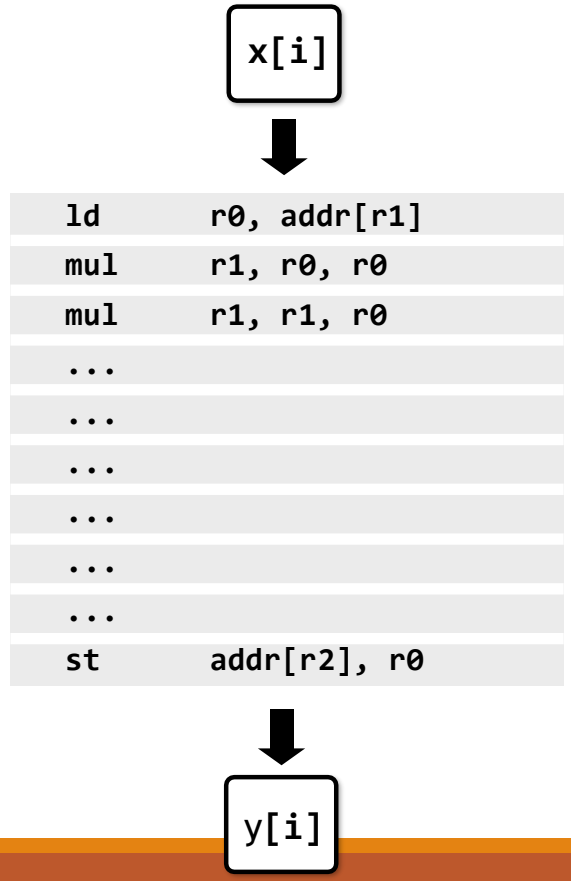
Same instruction broadcast to all ALUs
This operation is executed in parallel on all ALUs

Recall our original scalar program

```
void sinx(int N, int terms, float* x, float* y)
{
   for (int i=0; i<N; i++)
   {
      float value = x[i];
      float numer = x[i] * x[i] *
      int denom = 6;  // 3!     x[i];
      int sign = -1;

      for (int j=1; j<=terms; j++)
      {
         value  +=  sign  * numer   / denom;
         numer  *=  x[i]  * x[i];
         denom  *=  (2*j+2)   * (2*j+3);
         sign *= -1;
      }

      y[i] = value;
   }
}
```

Original compiled program:

Processes one array element using scalar instructions on scalar registers (e.g., 32-bit floats)



Vector program (using AVX intrinsics)

```
#include <immintrin.h>

void sinx(int N, int terms, float* x, float* y)
{
   float three_fact = 6;  // 3!
   for (int i=0; i<N; i+=8)
   {
      __m256 origx = _mm256_load_ps(&x[i]);
      __m256 value = origx;
      __m256 numer = _mm256_mul_ps(origx, _mm256_mul_ps(origx, origx));
      __m256 denom = _mm256_broadcast_ss(&three_fact);
      int sign = -1;

      for (int j=1; j<=terms; j++)
      {
         // value += sign * numer / denom
         __m256 tmp = _mm256_div_ps(_mm256_mul_ps(_mm256_set1ps(sign), numer), denom);
         value = _mm256_add_ps(value, tmp);

         numer = _mm256_mul_ps(numer, _mm256_mul_ps(origx, origx));
         denom = _mm256_mul_ps(denom, _mm256_broadcast_ss((2*j+2) * (2*j+3)));
         sign *= -1;
      }
      _mm256_store_ps(&y[i], value);
   }
}
```

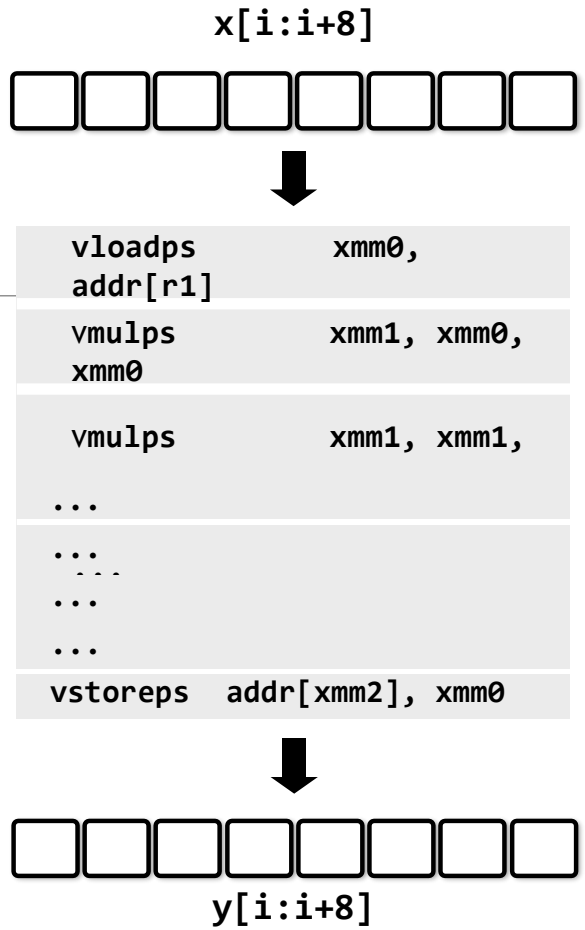
ZAIN UDDIN

Intrinsic datatypes and functions available to C programmers

Intrinsic functions operate on vectors of eight 32-bit values (e.g., vector of 8 floats)

Vector program (using AVX intrinsics)

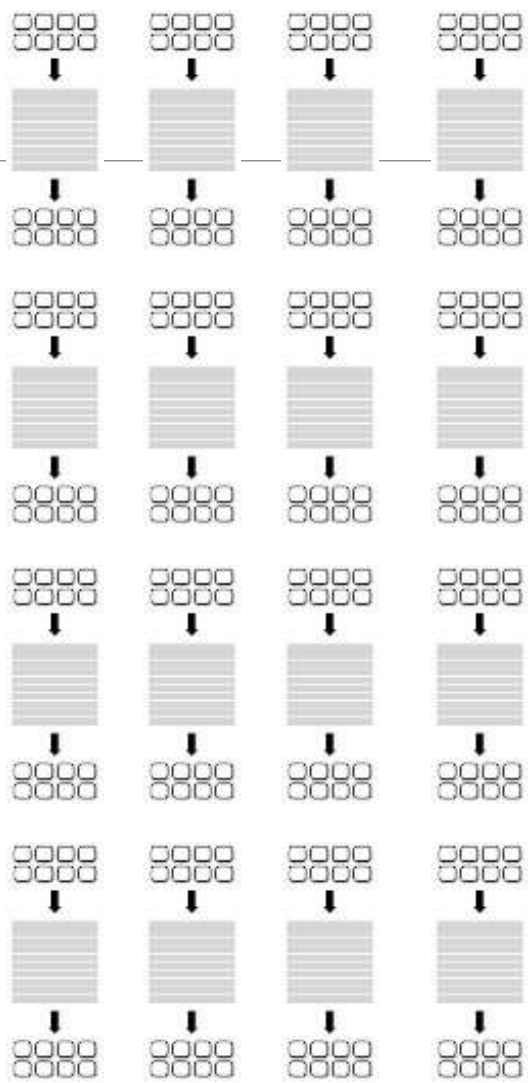
```
#include <immintrin.h>

void sinx(int N, int terms, float* x, float* y)
{
                                    !
   for (int i=0; i<N; i+=8)

      __m256 origx = _mm256_load_ps(&x[i]);
      __m256 value = origx;
      __m256 numer = _mm256_mul_ps(origx, _mm256_mul_ps(origx, origx));
      __m256 denom = _mm256_broadcast_ss(&three_fact);
      int sign = -1;

      for (int j=1; j<=terms; j++)
      {
         // value += sign * numer / denom
         __m256 tmp = _mm256_div_ps(_mm256_mul_ps(_mm256_set1ps(sign), numer), denom);
         value = _mm256_add_ps(value, tmp);

         numer = _mm256_mul_ps(numer, _mm256_mul_ps(origx, origx));
         denom = _mm256_mul_ps(denom, _mm256_broadcast_ss((2*j+2) * (2*j+3)));
         sign *= -1;
      }
      _mm256_store_ps(&y[i], value);
   }
}
```

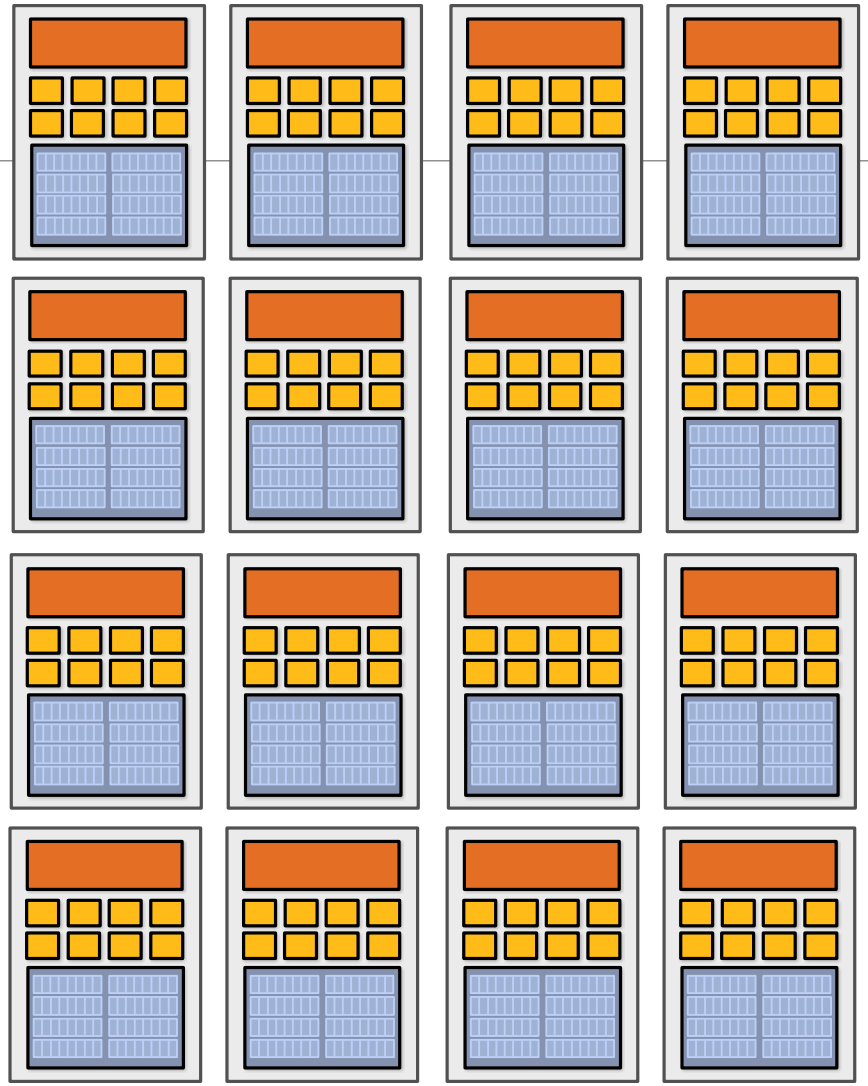


Compiled program:

Processes eight array elements simultaneously using vector instructions on 256-bit vector registers

16 SIMD cores: 128 elements in parallel





16 cores, 128 ALUs, 16 simultaneous instruction streams

Summary: three different forms of parallel execution

- **Superscalar:** exploit ILP within an instruction stream. Process different instructions from the <u>same</u> instruction stream in parallel (within a core)
  - Parallelism automatically discovered by the hardware during execution

- **SIMD:** multiple ALUs controlled by same instruction (within a core)
  - Efficient for data-parallel workloads: amortize control costs over many ALUs
  - Vectorization done by compiler (explicit SIMD) or at runtime by hardware (implicit SIMD)

- **Multi-core:** use multiple processing cores
  - Provides thread-level parallelism: simultaneously execute a completely different instruction
    stream on each core
  - Software creates threads to expose parallelism to hardware (e.g., via threading API)

But first let start with basic example with different flavors of parallelism

Let’s code an example of calculating the sum of squares for all elements in an array, demonstrating its transformation across different processing models

Array processing

```
#include <stdio.h>

int sum_of_squares_array(int data[], int len) {
  int sum = 0;
  for (int i = 0; i < len; ++i) {
    sum += data[i] * data[i];
  }
  return sum;
}

int main() {
  int data[] = {2, 5, 7, 4};
  int sum = sum_of_squares_array(data, 4);
  printf("Sum of squares: %d\n", sum);
  return 0;
}
```

- #include <stdio.h>: this line includes the standard input/output library for printing the result.
- int sum_of_squares_array(int data[], int len): Declares a function named sum_of_squares_array with two arguments: an integer array data and its length len. This function operates on the entire array simultaneously, a key characteristic of array processing.
- int sum = 0;: Initializes the sum variable to 0, similar to the SISD example.
- for (int i = 0; i < len; ++i): This loop iterates through each element in the data array, similar to the SISD example. However, the focus here isn't on individual elements but on the entire array as a whole.
- sum += data[i] * data[i];: This line remains the same as the SISD example, squaring each element and adding it to the sum. Again, although the loop iterates through each element, the operation affects the entire array's sum.
- return sum;: Returns the final sum of squares as the function's output.
- int main(): Same as the SISD example, this marks the program entry point.
- int data[] = {2, 5, 7, 4};: Defines the same sample data array.
- int sum = sum_of_squares_array(data, 4);: Calls the sum_of_squares_array function with the data array and its length, similar to the SISD example.
- printf("Sum of squares: %d\n", sum);: Prints the calculated sum to the console, just like before.
- return 0;: Indicates successful program termination.

Multithreading

```c
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>
typedef struct {
    int start;
    int end;
    int* data;
    int* partial_sum;
} thread_data_t;

void* worker(void* arg) {
    thread_data_t* data = (thread_data_t*)arg;
    int sum = 0;
    for (int i = data->start; i < data->end; ++i) {
        sum += data->data[i] * data->data[i];
    }
    int* result = (int*)malloc(sizeof(int));
    *result = sum;
    return result;
}
int sum_of_squares_multithread(int data[], int len, int num_threads)
{
    pthread_t threads[num_threads];
    thread_data_t thread_data[num_threads];
```

```c
for (int i = 0; i < num_threads; ++i) {
        thread_data[i].start = i * (len / num_threads);
        thread_data[i].end = (i == num_threads - 1) ? len : (i + 1) * (len / num_threads);
        thread_data[i].data = data;
        thread_data[i].partial_sum = NULL;
        pthread_create(&threads[i], NULL, worker, &thread_data[i]);
    }

    int total_sum = 0;
    for (int i = 0; i < num_threads; ++i) {
        void* result;
        pthread_join(threads[i], &result);
        total_sum += *(int*)result;
        free(result);
    }

    return total_sum;
}

int main() {
    int data[] = {2, 5, 7, 4};
    int sum = sum_of_squares_multithread(data, 4, 2);
    printf("Sum of squares: %d\n", sum);

    return 0;
}
```

Multithreading

Include Headers:
#include <stdio.h>: Standard input/output functions.
#include <pthread.h>: POSIX threads library for multithreading support.
#include <stdlib.h>: Standard library functions, including memory allocation (malloc).
Define Thread Data Structure:
thread_data_t: A structure to hold data specific to each thread.
start, end: Define the range of data each thread will process.
data: Pointer to the array of data.
partial_sum: Pointer to the partial sum computed by each thread.
Thread Function (worker):
worker: The function that each thread executes.
arg: Void pointer to the thread-specific data (thread_data_t).
sum: Local variable to compute the partial sum for the thread.
The thread iterates over its assigned range of data, squares each element, and adds to sum.
Allocates memory for an integer (result) and stores the partial sum in it.
Returns the result.
Multithreading Function (sum_of_squares_multithread):
Creates an array of threads (pthread_t threads[num_threads]) and an array of thread-specific data (thread_data_t thread_data[num_threads]).
Assigns the start, end, data array, and partial_sum for each thread.
Creates threads using pthread_create, passing the worker function and thread-specific data.
Waits for each thread to complete using pthread_join.
Accumulates the results and frees allocated memory.

Refrence

Reference: Flynn, M. J. (1972). Some computer organizations and their effectiveness. IEEE Transactions on Computers, 100(9), 948-960.